SLIIT
Discover Your Future

BSC (Hons) in Information Technology
Object Oriented Concepts – IT1050
Assignment 02

Year 01 Semester 02 **2024 - Feb – June**

Topic : **Online Recipe Management System**

Group No : **MLB_03.01_09**

Campus : **Malabe**

Submitted Date : **18/05/2024**

We declare that this is our own work and this Assignment does not incorporate without acknowledgment any material previously submitted by anyone else in SLIIT or any other university/Institute. And we declare that each one of us equally contributed to the completion of this Assignment.

Student Registration Number	Student Name	Contact Number
IT23183018	HIRUSHA D.G.A.D	077 2424521
IT23191006	COORAY Y.H.	070 6080877
IT23195202	HARSHANA L.L.E.	076 1541638
IT23189294	GUNATHILAKA H.D.I.	070 2310863
IT23190566	GANEGODA R.B.	077 0377131

Content...

01. Description of the Requirements

- All the registered users have access to log in to their accounts using the credentials they have entered when they signed up.
- Guest users who are new to the platform are able create an account for themselves by going through Registering Process.
- All the registered users and guest users are allowed browse through the website and view the recipes labeled as FREE.
- In order to access the Exclusive Recipes, a registered user has to become a premium user by purchasing a premium package.
- All the user have access to payment portal to make their payment transactions.
- Only a premium user grants the eligibility to become a Recipe Creator and publish recipes of his/her own.
- Only the registered users are allowed post reviews or feedbacks.
- Only the registered users and creators have access to the Blog but guest users are able to visit the Blog and read without any publications.
- All the users can issue their questions regarding the website through the Contact Us page.
- All creator should follow Terms & Conditions and Privacy Policies which are published in the website.
- Admin gets the permission to approve, edit, delete both user accounts and content that are published in the website.
- Manager has access to validate payments, generate reports and also calculate and issue discounts accordingly.
- A user will be granted 10% or 15% discounts according to the amounts they spend.

02. Identified Classes

- User
- GuestUser
- FreePlanUser
- PremiumUser
- PremiumMembership
- Emloyee
- Admin
- Manager
- RecipeCreator
- Recipe
- Wallet
- Report
- Feedback
- BlogPost
- Payment
- Discount

03. CRC Cards

GuestUser	
Responsibilities	**Collaborations**
Browse the website	
Create an Account	Admin

FreePlanUser	
Responsibilities	**Collaborations**
Login	
View Recipes	Recipe, RecipeCreator
Make Payments	Manager, Payment, Discount, PremiumMembership
Add a new Feedback	Admin, Creator
Edit the Profile	Admin
Add a new Blog Post	Admin

PremiumUser	
Responsibilities	**Collaborations**
extendMembership	Manager, Payment, Discount, PremiumMembership
Check exclusive recipes	
Become a Creator	Admin, Manager

Admin	
Responsibilities	**Collaborations**
Manage User Accounts	GuestUser, FreePlanUser, PremiumUser
Manage Contents	Recipe, Feedback, BlogPost

Manager	
Responsibilities	**Collaborations**
Generate Reports	Report
Calculate Discounts	Discount
Approve Payments	Payment, GuestUser, FreePlanUser, PremiumUser

RecipeCreator	
Responsibilities	**Collaborations**
Publish Recipes	Recipe
Read Feedbacks	Feedback
Post Free Recipes	GuestUser, FreePlanUser
Post Exclusive Recipes	PremiumUser
Withdraw Money	Wallet, Payment, Manager

Payment	
Responsibilities	**Collaborations**
Storing transaction details	Admin, Manager
Validation	Manager

Recipe	
Responsibilities	**Collaborations**
Storing Recipe Title	RecipeCreator
Storing Recipe Method	RecipeCreator
Approval form the publication	Admin
Being added to favourites	FreePlanUser, PremiumUser

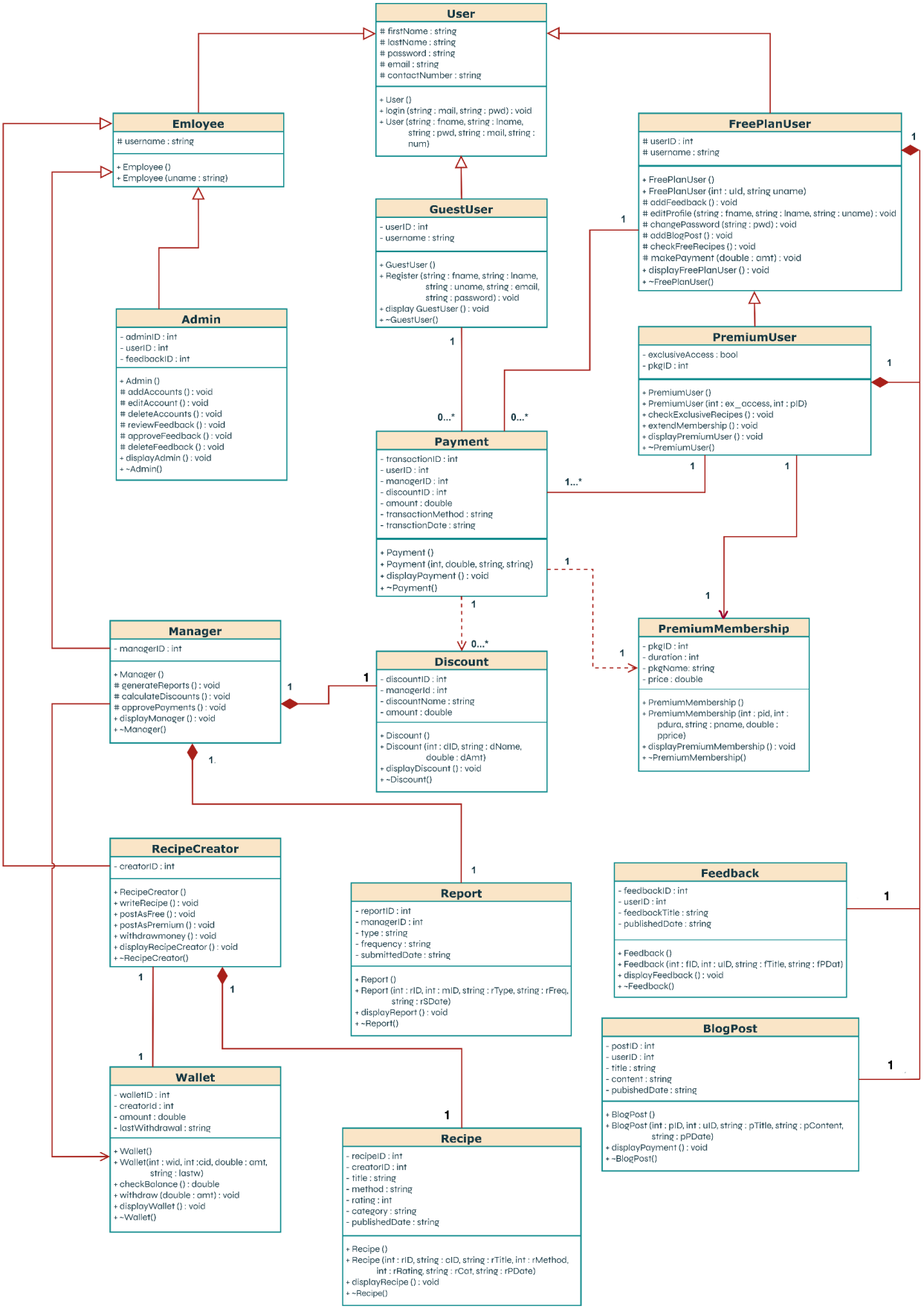
Report	
Responsibilities	**Collaborations**
Storing timely accourate details	Admin
Being generated in a relevance frequency	Manager

Feedback	
Responsibilities	**Collaborations**
Storing posted user details	FreePlanUser, PremiumUser
Getting approval	Admin

BlogPost	
Responsibilities	**Collaborations**
Storing posted user details	FreePlanUser, PremiumUser
Getting approval	Admin

Wallet	
Responsibilities	**Collaborations**
Storing relevant Creator’s details	RecipeCreator
Validating the transactions being made	Manager
Keep the balance up to date	Manager
Improved Security Measurements	Admin

04. Class Diagram



05. Coding for the identified classes

User.h

```
#include <iostream>
using namespace std;

class User {
    protected:
        string firstName;
        string lastName;
        string password;
        string email;
        string contactNumber;

    public:
        User();
        User(string fname, string lname, string pwd, string
mail, string contact);
        void login(string mail, string pwd);
        ~User();
};
```

User.cpp

```
#include <iostream>
#include "User.h"

User::User() {};

User::User(string fname, string lname, string pwd,
string mail, string contact) {
    firstName = fname;
    lastName = lname;
    password = pwd;
    email = mail;
    contactNumber = contact;
}

void User::login(string mail, string pwd) {}

User::~User() {}
```

Main.cpp

```cpp
#include <iostream>
#include "User.h"
#include "GuestUser.h"
#include "FreePlanUser.h"
#include "PremiumUser.h"
#include "Emloyee.h"
#include "Admin.h"
#include "Manager.h"
#include "RecipeCreator.h"
#include "Recipe.h"
#include "Wallet.h"
#include "Payment.h"
#include "Feedaback.h"
#include "Payment.h"
#include "BlogPost.h"
#include "PremiumMembership.h"
#include "Discount.h"

using namespace std;

int main() {
    // creating a new FreePlan object
    FreePlanUser* dineth = new FreePlanUser(1,"dineth@2002", "Dineth",
      Hirusha", "dineth@02", "dineth@gmail.com", "077-2424521", 1,
      "Incredible..!", "Very Delicious :)", "2024-05-17", 1, "Great
      Recipe");

    // passing the payment object to dineth's makePayment method
    dineth->makePayment(transaction1);

    // displaying the "dineth" object
    dineth->displayFreePlanUser();

    // creating a new Premium object
    PremiumUser* eshan = new PremiumUser(2, "eshan@2002", "Eshan",
      "Harshana", "eshan@02", "eshan@gmail.com", "076-1541638", true, 1,
      membership);

    // view the membership package that eshan has bought
    eshan->showMembership(membership);

    // displaying the "eshan" object
    eshan->displayPremiumUser();

    // creating new GuestUser object
    GuestUser* ranidu = new GuestUser();

    // passing the payment object to dinil's makePayment method
    dinil->makePayment(transaction2);

    // entering details to "ranidu" object
    ranidu->Register(3, "ranidu@2003", "Ranidu", "Ganegoda", "ranidu@02",
      "ranidu@gmail.com", "077-0377131")
```

```
// displaying the "ranidu" object
    ranidu->displayGuestUser();

    // creating a new Admin object
    Admin* dinil = new Admin(1, "dinil@2002", "Dinil", "Gunathilaka",
      "dinil@02", "dinil@gmail.com", "070-2310863");

    // displaying the "dinil" object
    dinil->displayAdmin();

    // creating a new Manager object
    Manager* yashodha = new Manager(1, "yashodha@2003", "Yashodha",
      "Cooray", "yashodha@02", "yashodha@gmail.com", "070-6080877", 1, 1,
      "Black Friday", 15.5, 1, 1, "Performance", "Monthly", "2024-05-
      17");

    // displaying "yashodha" object
    yashodha->displayManager();

    // creating a new Recipe Creator
    RecipeCreator* chef = new RecipeCreator(1, "chef@2002", "Chef",
      "Dias", "chef@02", "chef@gmail.com", "077-8921540", 1, 1,
      "Chocolate Chip Cookies", "1. Preheat oven to 350°F. 2. In a bowl,
      cream butter and sugars. 3. Beat in eggs and vanilla. 4. Mix in
      flour, baking soda, and salt. 5. Stir in chocolate chips. 6. Drop
      dough by rounded spoonfuls onto baking sheets. 7. Bake for 10-12
      minutes or until golden brown.", 9, "Snack", "2024-05-17");

    // displaying the "chef" object
    chef->displayRecipeCreator();

    // creating a new Membership object
    PremiumMembership* membership = new PremiumMembership(2, "SILVER",
12, 49.99);

    // displaying "membership" object
    membership->displayPremiumMembership();

    // creating a new Payment object
    Payment* transaction1 = new Payment(1, 49.99, "VISA", "5-13-2024",
      dineth);
    Payment* transaction2 = new Payment(2, 49.99, "VISA", "5-15-2024",
      dinil);

    // passing the "membership" object into calAfterPkgPayment method
    transaction1->calAfterPkgPayment(membership);
    transaction2->calAfterPkgPayment(membership);

    // creating a discount object
    Discount* discount = new Discount(1, 1, "Black Friday", 8.25);

    // passing the discount object into calAfterDiscount method
    transaction1->calAfterDiscount(discount);
    transaction2->calAfterDiscount(discount);
```

```
// creating a discount object
    Discount* discount = new Discount(1, 1, "Black Friday", 8.25);

    // passing the discount object into calAfterDiscount method
    transaction1->calAfterDiscount(discount);
    transaction2->calAfterDiscount(discount);

    // passing the "membership" object into calAfterPkgPayment method
    transaction1->calAfterPkgPayment(membership);
    transaction2->calAfterPkgPayment(membership);

    // displaying the "transaction" object
    transaction1->displayPayment();
    transaction2->displayPayment();

    // creating a new Wallet object
    Wallet* wally = new Wallet(1, 120.00, "5-13-2024", chef);

    // passing the wally object into Recipe Creator's view method
    chef->viewWallet(wally);

    //passing the wally object into Manager's manage method
    yashodha->manageWallet(wally);

    delete dineth;
    delete eshan;
    delete ranidu;
    delete dinil;
    delete yashodha;
    delete chef;
    delete membership;
    delete transaction1;
    delete transaction2;
    delete wally;

    return 0;
}
```

GuestUser.h

```
#include <iostream>
#include "User.h"

using namespace std;

class GuestUser : protected User {
    private:
        int userID;
        string username;

    public:
        GuestUser();
        void Register(int id, string uname, string fname, string
lname, string pwd, string mail, string contact) : User (fname,
lname, pwd, mail, contact);
        void displayGuestUser();
        ~GuestUser();
};
```

GuestUser.cpp

```
#include <iostream>
#include "User.h"
#include "GuestUser.h"

using namespace std;

Guestuser::GuestUser() {}

void GuestUser::Register(int id, string uname, string fname, string
lname, string pwd, string mail, string contact) : User (fname,
lname, pwd, mail, contact) {
    userID = id;
    username = uname;
};

void GuestUser::displayGuestUser() {
    cout << "User ID : " << userID << endl;
    cout << "Username : " << username << endl;
    cout << "First Name : " << firstName << endl;
    cout << "Last Name : " << lastName << endl;
    cout << "Email : " << email << endl;
    cout << "Password : **************" << endl;
    cout << "Contact Number : " << contactNumber << endl;
}

GuestUser::~GuestUser() {};
```

FreePlanUser.h

```
#include <iostream>
#include "User.h"
#include "Feedback.h"
#include "Payment.h"
#include "BlogPost.h"

class Feedback;
class BlogPost;
class Payment;

using namespace std;

class FreePlanUser : protected User{
    protected:
        int userID;
        string username;
        void addFeedback();
        void addBlogPost();
        void checkFreeRecipes();
        void changePassword(string pwd);
        void editProfile(string fname, string lname, string uname);
        void makePayment(double amt);

        Feedback* fb;
        BlogPost* bp;

        Payment* pmt;

    public:
        FreePlanUser();

        FreePlanUser(int id, string uname, string fname, string lname,
        string pwd, string mail, string contact) : User (fname, lname,
        pwd, mail, contact);

        FreePlanUser(int id, string uname, string fname, string lname,
        string pwd, string mail, string contact, int fid, string ftitle,
        string datetime) : User (fname, lname, pwd, mail, contact)

        FreePlanUser(int id, string uname, string fname, string lname,
        string pwd, string mail, string contact, int pid, string btitle,
        string bcontent, string date) : User (fname, lname, pwd, mail,
        contact)

        FreePlanUser(int id, string uname, string fname, string lname,
        string pwd, string mail, string contact, int pid, string btitle,
        string bcontent, string date, int fid, string ftitle);
```

```
        void addFeedback();
        void addBlogPost();
        void checkFreeRecipes();
        void changePassword(string pwd);
        void editProfile(string fname, string lname, string uname);
        void makePayment(Payment* pay);
        void displayFreePlanUser();
        ~FreePlanUser();
};
```

FreePlanUser.cpp

```
#include <iostream>
#include "User.h"
#include "FreePlanUser.h"

using namespace std;

FreePlanUser::FreePlanUser() {
    fb = new Feedback(0, 0, "Not set", "Not set");

    bp = new BlogPost(0, 0, "Not set", "Not set", "Not set");
}

FreePlanUser::FreePlanUser(int id, string uname, string fname,
string lname, string pwd, string mail, string contact) : User
(fname, lname, pwd, mail, contact) {
    userID = id;
    username = uname;
}

FreePlanUser::FreePlanUser(int id, string uname, string fname,
string lname, string pwd, string mail, string contact, int fid,
string ftitle, string datetime) : User (fname, lname, pwd, mail,
contact) {
    userID = id;
    username = uname;

    fb = new Feedback(fid, id, ftitle, datetime);
}

FreePlanUser::FreePlanUser(int id, string uname, string fname,
string lname, string pwd, string mail, string contact, int pid,
string btitle, string bcontent, string date) : User (fname, lname,
pwd, mail, contact) {
    userID = id;
    username = uname;

    bp = new BlogPost(pid, id, btitle, bcontent, date);
}

FreePlanUser::FreePlanUser(int id, string uname, string fname,
string lname, string pwd, string mail, string contact, int pid,
string btitle, string bcontent, string date, int fid, string
ftitle, string datetime) : User (fname, lname, pwd, mail, contact)
{
    userID = id;
    username = uname;

    bp = new BlogPost(pid, id, btitle, content, date);
    fb = new Feedback(fid, id, ftitle, date);
}
```

```
void FreePlanUser::addFeedback() {}

void FreePlanUser::addBlogPost() {}

void FreePlanUser::checkFreeRecipes() {}

void FreePlanUser::changePassword(string pwd) {}

void FreePlanUser::editProfile(string fname, string lname, string
uname) {}

void FreePlanUser::makePayment(Payment* pay) {
    pmt = pay;
}

void FreePlanUser::displayFreePlanUser() {
    cout << "User ID : " << userID << endl;
    cout << "Username : " << username << endl;
    cout << "First Name : " << firstName << endl;
    cout << "Last Name : " << lastName << endl;
    cout << "Email : " << email << endl;
    cout << "Password : **************" << endl;
    cout << "Contact Number : " << contactNumber << endl;
    fb->displayFeedback();
    bp->displayBlogPost();
}

FreePlanUser::~FreePlanUser() {
    delete fb;
    delete bp;
    delete pmt;
}
```

PremiumUser.h

```
#include <iostream>
#include "User.h"
#include "FreePlanUser.h"
#include "PremiumMembership.h"

using namespace std;

class PremiumUser : protected FreePlanUser {
    private:
        bool exclusiveAccess;
        int pkgID;

        PremiumMembership* pre_mem;

    public:
        PremiumUser();

        PremiumUser(int id, string uname, string fname, string lname,
string pwd, string mail, string contact, bool ex_acc, string pID) :
FreePlanUser (id, uname, fname, lname, pwd, mail, contact);

        void viewMembership(PremiumMembership* pmem);

        void checkExclusiveRecipes();

        void extendMembership();

        void displayPremiumUser();

        ~PremiumUser();
};
```

PremiumUser.cpp

```cpp
#include <iostream>
#include "User.h"
#include "FreePlanUser.h"
#include "PremiumUser.h"

using namespace std;

PremiumUser::PremiumUser() {};

PremiumUser::PremiumUser(int id, string uname, string fname, string
lname, string pwd, string mail, string contact, bool ex_acc, string
pID) : FreePlanUser (id, uname, fname, lname, pwd, mail, contact) {
    exclusiveAccess = ex_acc;
    pkgID = pID;
}

void PremiumUser::viewMembership(PremiumMembership* pmem) {
    pre_mem = pmem;
    pre_mem->displayPremiumMembership();
}

void PremiumUser::checkExclusiveRecipes() {}

void PremiumUser::extendMembership() {}

void PremiumUser::displayPremiumUser() {
    cout << "User ID : " << userID << endl;
    cout << "Username : " << username << endl;
    cout << "First Name : " << firstName << endl;
    cout << "Last Name : " << lastName << endl;
    cout << "Email : " << email << endl;
    cout << "Password : **************" << endl;
    cout << "Contact Number : " << contactNumber << endl;
}

PremiumUser::~PremiumUser() {
    delete pre_mem;
};
```

Employee.h

```
#include <iostream>
#include "User.h"

using namespace std;

class Employee : protected User {
    protected:
        string username;

    public:
        Employee();
        Employee(string uname, string fname, string lname, string pwd,
string mail, string contact) : User (fname, lname, pwd, mail, contact);
        ~Employee();
};
```

Employee.cpp

```
#include <iostream>
#include "User.h"
#include "Employee.h"

using namespace std;

Employee::Employee() {}

Employee::Employee(string uname, string fname, string lname, string
pwd, string mail, string contact) : User (fname, lname, pwd, mail,
contact) {
    username = uname;
}

Employee::~Employee() {}
```

Admin.h

```
#include <iostream>
#include "User.h"
#include "Employee.h"

class Admin : protected Employee {
    private:
        int adminID;
        int userID;
        int feedbackID;

    protected:
        void addAccount();
        void editAccount();
        void deleteAccount();
        void reviewFeedback();
        void approveFeedback();
        void deleteFeedback();

    public:
        Admin();

        Admin(int aid, string uname, string fname, string lname, string
         pwd, string mail, string contact) : Employee (uname, fname,
         lname, pwd, mail, contact);

        Admin(int aid, int uid, int fid, string uname, string fname,
        string lname, string pwd, string mail, string contact) : Employee
        (uname, fname, lname, pwd, mail, contact);

        void displayAdmin();

        ~Admin();
};
```

Admin.cpp

```cpp
#include <iostream>
#include "User.h"
#include "Employee.h"
#include "Admin.h"

using namespace std;

Admin::Admin() {}

Admin(int aid, string uname, string fname, string lname, string
pwd, string mail, string contact) : Employee (uname, fname, lname,
pwd, mail, contact) {
    adminID = aid;
}

Admin::Admin(int aid, int uid, int fid, string uname, string fname,
string lname, string pwd, string mail, string contact) : Employee
(uname, fname, lname, pwd, mail, contact) {
    adminID = aid;
    userID = uid;
    feedbackID = fid;
}

void Admin::addAccount() {}

void Admin::editAccount() {}

void Admin::deleteAccount() {}

void Admin::reviewFeedback() {}

void Admin::approveFeedback() {}

void Admin::deleteFeedback() {}

void Admin::displayAdmin() {
    cout << "Employee ID : " << empID << endl;
    cout << "Username : " << username << endl;
    cout << "First Name : " << firstName << endl;
    cout << "Last Name : " << lastName << endl;
    cout << "Email : " << email << endl;
    cout << "Password : **************" << endl;
    cout << "Contact Number : " << contactNumber << endl;
}

Admin::~Admin() {}
```

Manager.h

```
#include <iostream>
#include "User.h"
#include "Report.h"
#include "Wallet.h"
#include "Employee.h"

class Discount;
class Report;
class Wallet;

using namespace std;

class Manager : protected Employee {
    private:
        int managerID;

        Discount* dct;
        Report* rpt;

        Wallet* wallet;

    protected:
        void generateReport();
        void calculateDiscount();
        void approvePayment();

    public:
        Manager();

        Manager(int mid, string uname, string fname, string lname, string
         pwd, string mail, string contact)  : Employee (uname, fname,
        lname, pwd, mail, contact);

        Manager(int mid, string uname, string fname, string lname, string
         pwd, string mail, string contact, int did, string dname, double
         amt)  : Employee (uname, fname, lname, pwd, mail, contact);

        Manager(int mid, string uname, string fname, string lname, string
         pwd, string mail, string contact, int rid, string rtype, string
        frqcy, string datetime)  : Employee (uname, fname, lname, pwd,
         mail, contact);

        Manager(int mid, string uname, string fname, string lname, string
        pwd, string mail, string contact, int did, int manid, string
        dname, double amt, int rid, int mid, string rtype, string frqcy,
        string datetime, int wid, int cid, double amt, string
        datetime)  : Employee (uname, fname, lname, pwd, mail, contact);

        void ManageWallet(Wallet* wall);

        void displayManager();

        ~Manager();
};
```

Manager.cpp

```cpp
#include <iostream>
#include "User.h"
#include "Employee.h"
#include "Manager.h"
#include "Report.h"
#include "Discount.h"

using namespace std;

Manager::Manager() {
    dct = new Discount(0, 0, "Not set", 0);

    rpt = new Report(0, 0, "Not set", "Not set", "Not set");
}

Manager::Manager(int mid, string uname, string fname, string lname,
string pwd, string mail, string contact)  : Employee (uname, fname,
lname, pwd, mail, contact) {
    managerID = mid;
}

Manager::Manager(int mid, string uname, string fname, string lname,
string pwd, string mail, string contact, int rid, string rtype,
string frqcy, string datetime)  : Employee (uname, fname, lname,
pwd, mail, contact) {
    managerID = mid;

    rpt = new Report(rid, mid, rtype, frqcy, datetime);
}

Manager::Manager(int mid, string uname, string fname, string lname,
string pwd, string mail, string contact, int did, string dname,
double amt)  : Employee (uname, fname, lname, pwd, mail, contact) {
    managerID = mid;

    dct = new Discount(did, mid, dname, amt);
}

Manager::Manager(int mid, string uname, string fname, string lname,
string pwd, string mail, string contact, int did, int manid, string
dname, double amt, int rid, int mid, string rtype, string frqcy,
string datetime) : Employee (uname, fname, lname, pwd, mail,
contact) {
    managerID = mid;

    dct = new Discount(did, manid, dname, amt);

    rpt = new Report(rid, mid, rtype, frqcy, datetime);
}
```

```cpp
void Manager::ManageWallet(Wallet* wall) {
    wallet = wall;
}

void Manager::generateReport() {}

void Manager::calculateDiscount() {}

void Manager::approvePayment() {}

void Manager::displayManager() {
    cout << "Employee ID : " << empID << endl;
    cout << "Username : " << username << endl;
    cout << "First Name : " << firstName << endl;
    cout << "Last Name : " << lastName << endl;
    cout << "Email : " << email << endl;
    cout << "Password : **************" << endl;
    cout << "Contact Number : " << contactNumber << endl;
    dct->displayDiscount();
    rpt->displayReport();
}

Manager::~Manager() {
    delete dct;
    delete rpt;
    delete wallet;
};
```

RecipeCreator.h

```
#include <iostream>
#include "User.h"
#include "Recipe.h"
#include "Wallet.h"
#include "Employee.h"

using namespace std;

class Wallet;
class Recipe;

class RecipeCreator : protected Employee {
    private:
        int creatorId;

        Recipe* rcp;

        Wallet* wallet;

    public:
        RecipeCreator();

        RecipeCreator(int id, string uname, string fname, string lname,
string pwd, string mail, string contact, int rid, int cid, string rTitle,
string rMethod, int rRating, string rCat, string rPDate, Wallet* wallet)
: Employee (uname, fname, lname, pwd, mail, contact);

        void writeRecipe(string title, string method);

        void postAsFree();

        void postAsPremium();

        void viewWallet(Wallet* wall);

        void withdrawMoney(double amt);

        void displayRecipeCreator();

        ~RecipeCreator();
};
```

RecipeCreator.cpp

```cpp
#include <iostream>
#include "User.h"
#include "Recipe.h"
#include "Wallet.h"
#include "Employee.h"
#include "RecipeCreator.h"

using namespace std;

RecipeCreator::RecipeCreator() {
    rcp = new Recipe(0, 0, "Not set", "Not set", 0, "Not set", "Not
set");
}

RecipeCreator::RecipeCreator(int id, string uname, string fname, string
lname, string pwd, string mail, string contact, int rid, int cid, string
rTitle, string rMethod, int rRating, string rCat, string rPDate) :
Employee (uname, fname, lname, pwd, mail, contact) {
    creatorId = id;

    rcp = new Recipe(rid, cid, rTitle, rMethod, rRating, rCat, rPDate);
}

void RecipeCreator::writeRecipe(string title, string method) {}

void RecipeCreator::postAsFree() {}

void RecipeCreator::postAsPremium() {}

void RecipeCreator::viewWallet(Wallet* wall) {
    wallet = wall;
    wallet->displayWallet();
}

void RecipeCreator::withdrawMoney(double amt) {}

void RecipeCreator::displayRecipeCreator() {
    cout << "Employee ID : " << empID << endl;
    cout << "Username : " << username << endl;
    cout << "First Name : " << firstName << endl;
    cout << "Last Name : " << lastName << endl;
    cout << "Email : " << email << endl;
    cout << "Password : **************" << endl;
    cout << "Contact Number : " << contactNumber << endl;
    rcp->displayRecipe();
    wallet->displayWallet();
}

RecipeCreator::~RecipeCreator() {
    delete rcp;
    delete wallet;
}
```

PremimMembership.h

```
#include <iostream>

using namespace std;

class PremiumMembership {
    private:
        int pkgID;
        string pkgName;
        int duration;
        double price;

    public:
        PremiumMembership();
        PremiumMembership(int id, string pname, int pduration, double
pprice);
        void displayPremiumMembership;
        ~PremiumMembership()
};
```

PremiumMembership.cpp

```cpp
#include <iostream>
#include "PremiumMembership.h"

using namespace std;

PremiumMembership::PremiumMembership() {}

PremiumMembership::PremiumMembership(int id, string pname, int
pduration, double pprice) {
    pkgID = id;
    pkgName = pname;
    duration = pduration;
    price = pprice;
}

void displayPremiumMembership() {
    cout << "Package ID : " << pkgID << endl;
    cout << "Package Name : " << pkgName << endl;
    cout << "Duration : " << duration << endl;
    cout << "Price : " << price << endl;
}

PremiumMembership::~PremiumMembership() {}
```

PremiumUser.h

```
#include <iostream>
#include "User.h"
#include "FreePlanUser.h"
#include "PremiumMembership.h"

using namespace std;

class PremiumUser : protected FreePlanUser {
    private:
        bool exclusiveAccess;
        int pkgID;

        PremiumMembership* pre_mem;

    public:
        PremiumUser();

        PremiumUser(int id, string uname, string fname, string
lname, string pwd, string mail, string contact, bool ex_acc, string
pID) : FreePlanUser (id, uname, fname, lname, pwd, mail, contact);

        void viewMembership(PremiumMembership* pmem);

        void checkExclusiveRecipes();

        void extendMembership();

        void displayPremiumUser();

        ~PremiumUser();
};
```

PremiumUser.cpp

```cpp
#include <iostream>
#include "User.h"
#include "FreePlanUser.h"
#include "PremiumUser.h"

using namespace std;

PremiumUser::PremiumUser() {};

PremiumUser::PremiumUser(int id, string uname, string fname, string
lname, string pwd, string mail, string contact, bool ex_acc, string
pID) : FreePlanUser (id, uname, fname, lname, pwd, mail, contact) {
    exclusiveAccess = ex_acc;
    pkgID = pID;
}

void PremiumUser::viewMembership(PremiumMembership* pmem) {
    pre_mem = pmem;
    pre_mem->displayPremiumMembership();
}

void PremiumUser::checkExclusiveRecipes() {}

void PremiumUser::extendMembership() {}

void PremiumUser::displayPremiumUser() {
    cout << "User ID : " << userID << endl;
    cout << "Username : " << username << endl;
    cout << "First Name : " << firstName << endl;
    cout << "Last Name : " << lastName << endl;
    cout << "Email : " << email << endl;
    cout << "Password : **************" << endl;
    cout << "Contact Number : " << contactNumber << endl;
}

PremiumUser::~PremiumUser() {
    delete pre_mem;
};
```

Payment.h

```
#include <iostream>
#include "FreePlanUser.h"
#include "PremiumUser.h"
#include "GuestUser.h"
#include "Discount.h"
#include "PremiumMembership.h"

using namespace std;

class FreePlanUser;
class GuestUser;
class PremiumUser;
class Discount;
class PremiumMembership;

class Payment {
    private:
        int transactionID;
        int userID;
        int managerID;
        double amount;
        string transactionMethod;
        string transactionDate;

        FreePanUser* fpu;
        PremiumUser* pu;
        GuestUser* gu;

    public:
        Payment();

        Payment(int tid, int usid, int mid, double amt, string tmethod,
string tdate, FreePlanUser* free);

        Payment(int tid, int usid, int mid, double amt, string tmethod,
string tdate, FreePlanUser* pre);

        Payment(int tid, int usid, int mid, double amt, string tmethod,
string tdate, FreePlanUser* guest);

        calAfterDisPayment(Discount* dis);

        calAfterPkgPayment(PremiumMembership* mem);

        void displayPayment();

        ~Payment();
}
```

Payment.cpp

```cpp
#include <iostream>
#include "Payment.h"
#include "Discount.h"
#include "PremiumMembership.h"

using namespace std;

Payment::Payment() {}

Payment::Payment(int tid, int uid, int mid, double amt, string
tmethod, string tdate, FreePlanUser* free) {
    transactionID = id;
    userID = uid;
    managerID = mid;
    amount = amt;
    transactionMethod = tmethod;
    transactionDate = tdate;

    fpu = free;
}

Payment::Payment(int tid, int uid, int mid, double amt, string
tmethod, string tdate, PremiumUser* pre) {
    transactionID = id;
    userID = uid;
    managerID = mid;
    amount = amt;
    transactionMethod = tmethod;
    transactionDate = tdate;

    pu = pre;
}

Payment::Payment(int tid, int uid, int mid, double amt, string
tmethod, string tdate, GuestUser* guest) {
    transactionID = id;
    userID = uid;
    managerID = mid;
    amount = amt;
    transactionMethod = tmethod;
    transactionDate = tdate;

    gu = guest;
}
```

```cpp
Payment::calAfterDisPayment(Discount* dis) {
    this->amount -= dis->amount;
}

Payment::calAfterPkgPayment(PremiumMembership* mem) {
    this->amount = mem->price;
}

void Payment::displayPayment() {
    cout << "Transaction ID : " << transactionID << endl;
    cout << "Transaction Amount : " << amount << endl;
    cout << "Transaction Method : " << transactionMethod << endl;
    cout << "Transaction Date : " << transactionDate << endl;
}

Payment::~Payment() {
    delete fpu;
    delete pu;
    delete gu;
}
```

Discount.h

```
#include <iostream>
#include "Manager.h"
#include "Report.h"

using namespace std;

class Discount {
    private:
        int discountID;
        int managerID;
        string discountName;
        double amount;

    public:
        Discount();
        Discount(int id, int mid, string dname, double amt);
        void displayDiscount();
        ~Discount();
}
```

Discount.cpp

```
#include <iostream>
#include "Discount.h"
#include "Manager.h"
#include "Report.h"

using namespace std;

Discount();

Discount(int id, int mid, string dname, double amt) {
    discountID = id;
    managerID = mid
    discountName = dname;
    amount = amt;
}

void displayDiscount() {
    cout << "Discount ID : " << discountID << endl;
    cout << "Manager ID : " << managerID << endl;
    cout << "Discount Name : " << discountName << endl;
    cout << "Amount : " << amount << endl;
}

Discount::~Discount() {}
```

Report.h

```
#include <iostream>
#include "Discount.h"
#include "Manager.h"

using namespace std;

class Report {
    private:
        int reportID;
        int managerID;
        string type;
        string frequency;
        string submittedDate;

    public:
        Report();
        Report(int rid, int mid, string rtype, string frqcy, string
         datetime);
        void displayReport();
        ~Report();
}
```

Report.cpp

```
#include <iostream>
#include "Report.h"
#include "Discount.h"
#include "Manager.h"

using namespace std;

Report::Report() {}

Report::Report(int rid, int mid, string rtype, string frqcy, string
datetime) {
    reportID = rid;
    managerID = mid;
    type = rtype;
    frequency = frqcy;
    submittedDate = datetime;
}

void Report::displayReport() {
    cout << "Report ID : " << reportID << endl;
    cout << "Manager ID : " << managerID << endl;
    cout << "Report Type : " << type << endl;
    cout << "Frequency : " << frequency << endl;
    cout << "Submitted Date : " << submittedDate << endl;
}

Report::~Report() {}
```

Feedback.h

```
#include <iostream>

using namespace std;

class Feedback {
    private:
        int feedbackID;
        int userID;
        string feedbackTitle;
        string feedbackBody;
        string publishedDate;


    public:
        Feedback();
        Feedback(int fid, int u_id, string ftitle, string datetime);
        void displayFeedback();
}
```

Feedback.cpp

```
#include <iostream>
#include "Feedback.h"

using namespace std;

Feedback::Feedback() {}

Feedback::Feedback(int fid, int uid, string ftitle, string
datetime) {
    feedbackID = fid;
    userID = uid;
    feedbackTitle = ftitle;
    publishedDate = datetime;
}

void Feedback::displayFeedback() {
    cout << "Feedback ID : " << feedbackID << endl;
    cout << "User ID : " << userID << endl;
    cout << "Title : " << feedbackTitle << endl;
    cout << "Published Date : " << publishedDate << endl;
}
```

BlogPost.h

```
#include <iostream>

using namespace std;

class BlogPost {
    private:
        int postID;
        int userID;
        string title;
        string content;
        string publishedDate;

    public:
        BlogPost();
        BlogPost(int pid, int uid, string btitle, string bcontent, string
         date);
        void displayBlogPost();
        ~BlogPost();
}
```

BlogPost.cpp

```
#include <iostream>
#include "BlogPost.h"

using namespace std;

BlogPost::BlogPost() {}
BlogPost::BlogPost(int pid, int uid, string btitle, string
bcontent, string date) {
    postID = fid;
    userID = uid;
    title = btitle;
    content = bcontent;
    publishedDate = date;
}
void BlogPost::displayBlogPost() {
    cout << "Blog Post ID : " << postID << endl;
    cout << "User ID : " << userID << endl;
    cout << "Title : " << title << endl;
    cout << "Content : " << content << endl;
    cout << "Published Date : " << publishedDate << endl;
}

BlogPost::~BlogPost() {}
```

Recipe.h

```
#include <iostream>
#include "RecipeCreator.h"

using namespace std;

class Recipe {
    private:
        int recipeID;
        int creatorID;
        string title;
        string method;
        int rating;
        string category;
        string publishedDate;

    public:
        Recipe();
        Recipe(int rid, int cid, string rTitle, string rMethod, int
rRating, string rCat, string rPDate);
        void displayRecipe();
        ~Recipe();
};
```

Recipe.cpp

```cpp
#include <iostream>
#include "Recipe.h"
#include "RecipeCreator.h"

using namespace std;

Recipe::Recipe() {}

Recipe::Recipe(int rid, int cid, string rTitle, string rMethod, int
rRating, string rCat, string rPDate) {
    recipeID = rid;
    creatorID = cid;
    title = rTitle;
    method = rMethod;
    rating = rRating;
    category = rCat;
    publishedDate = rPDate;
}

void displayRecipe() {
    cout << "Recipe ID : " << recipeID << endl;
    cout << "Creator ID : " << creatorID << endl;
    cout << "Title : " << title << endl;
    cout << "Method : " << method << endl;
    cout << "Rating : " << rating << "/10" << endl;
    cout << "Category : " << category << endl;
    cout << "Published Date : " << publishedDate << endl;
}

Recipe::~Recipe() {}
```

Wallet.h

```
#include <iostream>

using namespace std;

class Wallet {
    private:
        int walletID;
        int creatorID;
        double balance;
        string lastWithdrawl;

        RecipeCreator* rcre;

    public:
        Wallet();
        Wallet(int wid, int cid, double amt, string datetime);
        Wallet(int wid, int cid, double amt, string datetime,
RecipeCreator* creator);
        double checkBalance();
        void withdraw(double amt);
        void displayWallet();
        ~Wallet();
};
```

Wallet.cpp

```cpp
#include <iostream>
#include "Wallet.h"

using namespace std;

Wallet::Wallet() {
    walletID = 0;
    creatorID = 0;
    balance = 0;
    lastWithdrawal = "Not Set";
}

Wallet::Wallet(int wid, int cid, double amt, string datetime) {
    walletID = wid;
    creatorID = cid;
    balance = amt;
    lastWithdrawal = datetime;
}

Wallet::Wallet(int wid, int cid, double amt, string datetime,
RecipeCreator* creator) {
    walletID = wid;
    creatorID = cid;
    balance = amt;
    lastWithdrawal = datetime;

    rcre = creator;
}

double Wallet::checkBalance() {}

void Wallet::withdraw(double amt) {}

void Wallet::displayWallet() {
    cout << "Wallet ID : " << walletID << endl;
    cout << "Balance : " << balance << endl;
    cout << "Last Withdrawal : " << lastwithdrawal << endl;
}

Wallet::~Wallet() {
    delete rcre;
}
```

06. Individual Contribution

IT23183018 : HIRUSHA D G A D

- Drew the class diagram using draw.io platform.
- Wrote the Description of the Requirements Section.
- Designed the Classes..,
  - RecipeCreator
  - Wallet
  - Recipe
  - FreePlanUser
- Coded for the above mentioned Classes.
- Created the CRC Cards for the relevant Classes.
- Coded the relevant part in the main.cpp file.
- Finalized the document by putting together everyone's codes.

IT23191006 : COORAY Y H

- Designed the Classes..,
  - Manager
  - Discount
  - Payment
- Coded for the above mentioned Classes.
- Created the CRC Cards for the relevant Classes.
- Coded the relevant part in the main.cpp file..

IT23195202 : HARSHANA L L E

- Designed the Classes..,
  - PremimUser
  - GuestUser
  - Feedback
- Coded for the above mentioned Classes.
- Created the CRC Cards for the relevant Classes.
- Coded the relevant part in the main.cpp file..

IT23189294 : GUNATHILAKA H D I

- Designed the Classes..,
  - User
  - Employee
  - Report
- Coded for the above mentioned Classes.
- Created the CRC Cards for the relevant Classes.
- Coded the relevant part in the main.cpp file..

IT23190566 : GANEGODA R B

- Designed the Classes..,
  - BlogPost
  - PremiumMembership
  - Admin
- Coded for the above mentioned Classes.
- Created the CRC Cards for the relevant Classes.
- Coded the relevant part in the main.cpp file..